

Lecture 7

Medians and Order Statistics



Slides are based on the textbook and its notes

Overview

- ❑ *i th order statistic* is the i th smallest element of a set of n elements.
- ❑ The *minimum* is the first order statistic ($i = 1$).
- ❑ The *maximum* is the n th order statistic ($i = n$).
- ❑ A *median* is the “halfway point” of the set.
- ❑ When n is odd, the median is unique, at $i = (n + 1)/2$.
- ❑ When n is even, there are two medians:
 - The *lower median*, at $i = n/2$, and
 - The *upper median*, at $i = n/2 + 1$.
 - We mean lower median when we use the phrase “the median.”

The selection problem

Input: A set of n distinct numbers and a number i , with $1 \leq i \leq n$.

Output: The element $x \in A$ that is larger than exactly $i - 1$ other elements in A . In other words, the i th smallest element of A .

- Easy to solve the selection problem in $O(n \lg n)$ time:
 - Sort the numbers using an $O(n \lg n)$ -time algorithm, such as heapsort or merge sort.
 - Then return the i th element in the sorted array.
- There are faster algorithms, however.
 - First, we will look at the problem of selecting the minimum and maximum of a set of elements.
 - Then, we will look at a simple general selection algorithm with a time bound of $O(n)$ in the average case.

Minimum and maximum

- We can easily obtain an upper bound of $n - 1$ comparisons for finding the minimum of a set of n elements.
- Examine each element in turn and keep track of the smallest one. The following pseudocode finds the minimum element in array $A[1 : n]$:

MINIMUM(A, n)

```
1   $min = A[1]$ 
2  for  $i = 2$  to  $n$ 
3      if  $min > A[i]$ 
4           $min = A[i]$ 
5  return  $min$ 
```

- The maximum can be found in exactly the same way by replacing the $>$ with $<$ in the above algorithm.

Simultaneous minimum and maximum

- Some applications need both the minimum and maximum of a set of elements.
 - For example, a graphics program may need to scale a set of (x, y) data to fit onto a rectangular display. To do so, the program must first find the minimum and maximum of each coordinate.
- A simple algorithm to find the minimum and maximum is to find each one independently. There will be $n - 1$ comparisons for the minimum and $n - 1$ comparisons for the maximum, for a total of $2n - 2$ comparisons. This will result in $\Theta(n)$ time. In fact, at most $3\lfloor n/2 \rfloor$ comparisons suffice to find both the minimum and maximum:

Simultaneous minimum and maximum

- Maintain the minimum and maximum of elements seen so far.
 - Do not compare each element to the minimum and maximum separately.
 - Process elements in pairs.
 - Compare the elements of a pair to each other.
 - Then compare the larger element to the maximum so far, and compare the smaller element to the minimum so far.
- This leads to only three comparisons for every two elements.
- Setting up the initial values for the min and max depends on whether n is odd or even.
- If n is even, compare the first two elements and assign the larger to max and the smaller to min. Then process the rest of the elements in pairs.
 - If n is odd, set both min and max to the first element. Then process the rest of the elements in pairs.

Simultaneous minimum and maximum

- *Analysis of the total number of comparisons*
- If n is even, do one initial comparison and then $3(n - 2)/2$ more comparisons.

$$\begin{aligned}\text{Number of comparisons} &= \frac{3(n - 2)}{2} + 1 \\ &= \frac{3n - 6}{2} + 1 \\ &= \frac{3n}{2} - 3 + 1 \\ &= \frac{3n}{2} - 2.\end{aligned}$$

- If n is odd, do $3(n - 1)/2 = 3\lfloor n/2 \rfloor$ comparisons.
- In either case, the maximum number of comparisons is $\leq 3\lfloor n/2 \rfloor$.

Selection in expected linear time: Algorithm

- Selection of the i th smallest element of the array A can be done in $\Theta(n)$ time.
- The procedure RANDOMIZED-SELECT uses RANDOMIZED-PARTITION from the quicksort algorithm. RANDOMIZED-SELECT differs from quicksort because it recurses on one side of the partition only.

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$       // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```


Selection in expected linear time: Algorithm

- After the call to RANDOMIZED-PARTITION, the array is partitioned into two subarrays $A[p : q - 1]$ and $A[q + 1 : r]$, along with a **pivot** element $A[q]$.
 - The elements of subarray are all $A[p : q - 1] \leq A[q]$.
 - The elements of subarray are all $A[q + 1 : r] > A[q]$.
 - The pivot element is the k th element of the subarray $A[p : r]$, where $k = q - p + 1$.
 - If the pivot element is the i th smallest element (i.e., $i = k$), return $A[q]$.
 - Otherwise, recurse on the subarray containing the i th smallest element.
 - If $i < k$, this subarray is $A[p : q - 1]$, and we want the i th smallest element.
 - If $i > k$, this subarray is $A[q + 1 : r]$ and, since there are k elements in $A[p : r]$ that precede $A[q + 1 : r]$, we want the $(i - k)$ th smallest element of this subarray.

Selection in expected linear time: Example

		p	r	i	partitioning																														
$A^{(0)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>6</td><td>19</td><td>4</td><td>12</td><td>14</td><td>9</td><td>15</td><td>7</td><td>8</td><td>11</td><td>3</td><td>13</td><td>2</td><td>5</td><td>10</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	6	19	4	12	14	9	15	7	8	11	3	13	2	5	10	1	15	5	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																					
6	19	4	12	14	9	15	7	8	11	3	13	2	5	10																					
$A^{(1)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>6</td><td>4</td><td>12</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>3</td><td>13</td><td>2</td><td>5</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	6	4	12	10	9	7	8	11	3	13	2	5	14	19	15	1	12	5	1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																					
6	4	12	10	9	7	8	11	3	13	2	5	14	19	15																					
$A^{(2)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>6</td><td>13</td><td>5</td><td>12</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	10	9	7	8	11	6	13	5	12	14	19	15	4	12	2	2
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																					
3	2	4	10	9	7	8	11	6	13	5	12	14	19	15																					
$A^{(3)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>6</td><td>12</td><td>5</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	10	9	7	8	11	6	12	5	13	14	19	15	4	11	2	3
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																					
3	2	4	10	9	7	8	11	6	12	5	13	14	19	15																					
$A^{(4)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>11</td><td>9</td><td>12</td><td>10</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	5	6	7	8	11	9	12	10	13	14	19	15	4	5	2	4
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																					
3	2	4	5	6	7	8	11	9	12	10	13	14	19	15																					
$A^{(5)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>11</td><td>9</td><td>12</td><td>10</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	5	6	7	8	11	9	12	10	13	14	19	15	5	5	1	5
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																					
3	2	4	5	6	7	8	11	9	12	10	13	14	19	15																					

In this example, we look for $i = 5$ th smallest element in $A[1 : 15]$. The action of RANDOMIZED-SELECT as successive partitionings narrow the subarray $A[p : r]$, showing the values of the parameters p , r , and i at each recursive call. The subarray $A[p : r]$ in each recursive step is shown in tan, with the dark tan element selected as the pivot for the next partitioning. Blue elements are outside $A[p : r]$. The answer is the tan element in the bottom array, where $p = r = 5$ and $i = 1$. The array designations $A^{(0)}$, $A^{(1)}$, ..., $A^{(5)}$, the partitioning numbers.

Selection in expected linear time: Performance

❑ *Worst-case running time*

Because we could be extremely unlucky and always recurse on a subarray that is only one element smaller than the previous subarray. Thus the running time is the same as for QUICKSORT: $T(n) = T(n - 1) + \Theta(n)$, with the solution $T(n) = \Theta(n^2)$.

❑ *Expected running time*

RANDOMIZED-SELECT works well on average. Because it is randomized, no particular input brings out the worst-case behavior consistently. The expected running time of RANDOMIZED-SELECT is $\Theta(n)$. Rigorous analysis can be found in the textbook on pages 232 ~ 236.

Reading

- ▣ Sections 9.1 ~ 9.2